Voice Scribe v2
Local-First Speech-to-Text for Development Workflows

Johnraymond Yanez

Voice Scribe

April 4, 2026

Overview

- **Motivation** — Why local transcription for developer workflows?
- **Architecture** — Two-service system over localhost
- **Container Strategy** — GPU passthrough, model caching, isolation
- **Audio Pipeline** — Browser recording through inference
- **Frontend** — React component tree and state machine
- **Security** — Threat model, privacy, operational telemetry
- **Development** — Workflow, tooling, and testing
- **Roadmap, v2.0+** — Milestones and what just shipped

The Problem

Developer workflows increasingly involve AI coding assistants — tools where natural language is the primary input.

Friction points

- Typing long prompts breaks flow
- Context-switching between thinking and typing
- Cloud transcription raises privacy concerns
- Existing tools not optimized for dev use cases

What we want

- Speak naturally, get text
- Text stays on your machine
- One-click copy into any tool
- No accounts, no cloud, no latency

Core Insight

Speech is the fastest way to externalize a thought. The bottleneck is getting that speech into text *locally* with sufficient accuracy.

Design Principles

Four principles govern every decision:

1. **Local-first**
   Audio and transcripts stay on the user's machine. No cloud services required at any point.
2. **Host for interaction, container for inference**
   The browser owns mic access, clipboard, and editing. The container owns model loading and GPU inference.
3. **Audio is transient, text is durable**
   Raw audio is short-lived input — deleted after processing. The transcript is the user-facing artifact.
4. **Scope matches the milestone**
   Repo structure and services reflect current reality, not future aspirations. Architecture stays flat until complexity is earned.

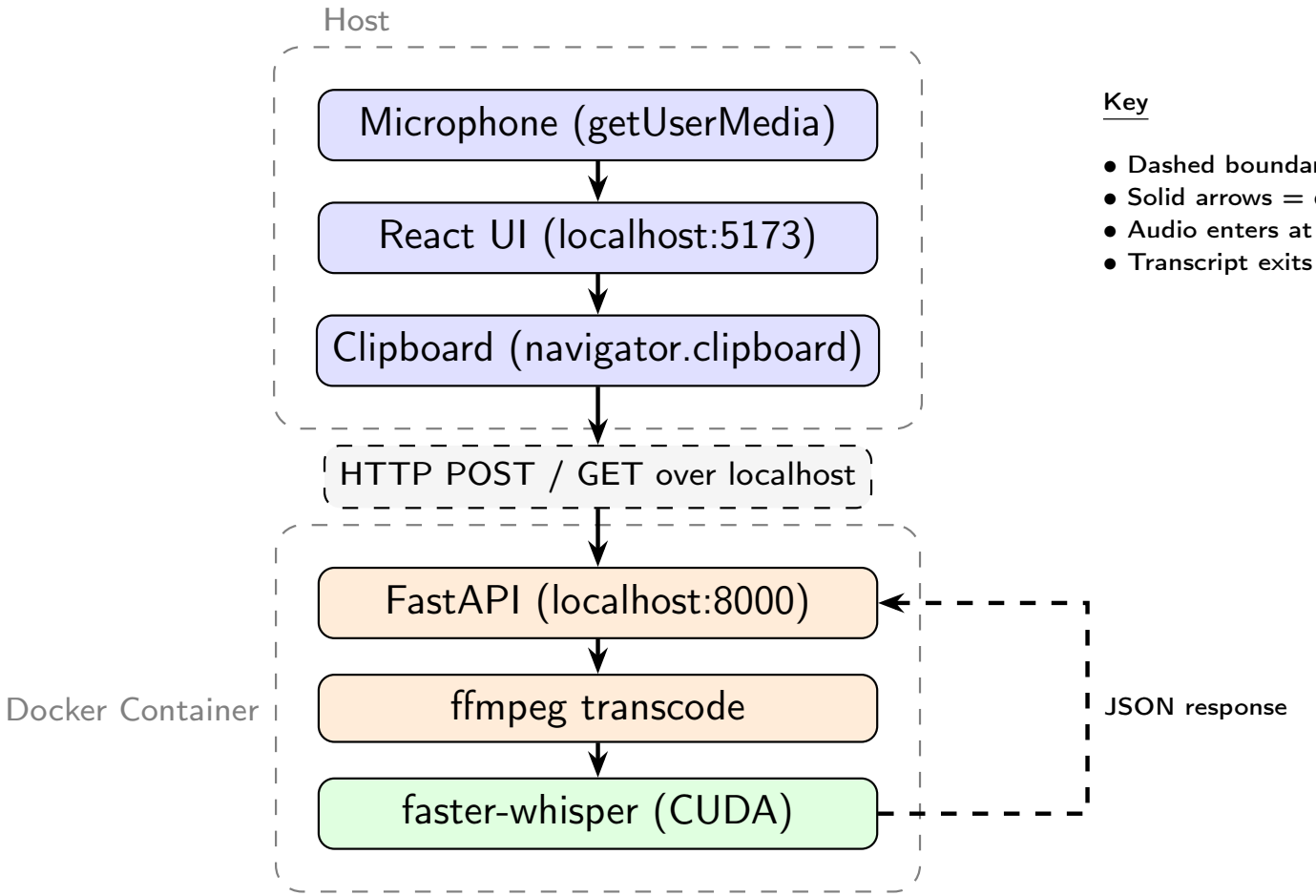
System Architecture

Voice Scribe is a two-service system communicating over localhost HTTP.

- The **frontend** runs on the host — it owns browser APIs (microphone, clipboard, DOM) that cannot exist inside a container
- The **backend** runs in a Docker container — it owns GPU-accelerated inference via CUDA, which cannot run in a browser
- Communication is plain HTTP. No WebSocket, no reverse proxy, no message queue
- Both services bind to `localhost` only — never network-accessible

The split is dictated by hardware constraints, not architectural preference. Each service does what only it can do.

System Architecture — Diagram



Tech Stack

Layer	Technology	Rationale
Frontend	React, TypeScript, Vite	Mic access, clipboard, fast iteration
Backend	Python, FastAPI	Model ecosystem, easy containerization
STT Engine	faster-whisper (large-v3)	Accuracy/speed on local GPU
Transcode	ffmpeg (in container)	Normalizes browser audio to WAV
Container	Docker, CUDA 12.6	GPU isolation, reproducibility

Selection Principle

Each layer is selected for its ecosystem strength at that boundary. React for browser APIs, Python for ML inference, Docker for GPU isolation. No layer is forced to do work outside its strength.

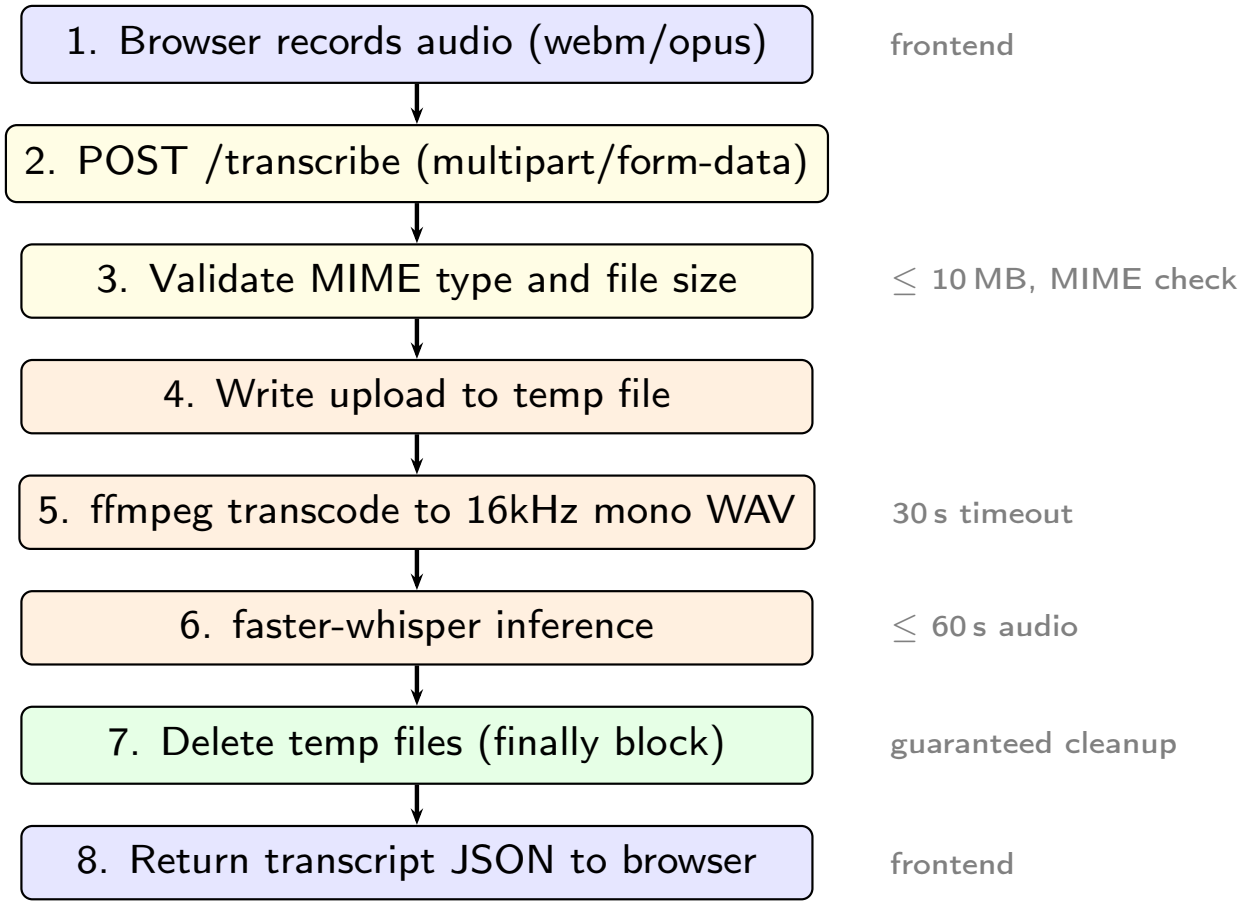
Request Lifecycle

A single transcription request passes through eight stages. The frontend handles stages 1 and 8; the backend handles 2–7.

- **Validation** (stages 2–3) rejects bad input early — before any expensive I/O or computation
- **Processing** (stages 4–6) is the core pipeline: temp file, transcode, inference
- **Cleanup** (stage 7) is guaranteed by `try/finally` — both temp files are deleted on success *and* failure
- Every stage between 3–7 is bounded by explicit limits: 10 MB upload, 60 s duration, 30 s ffmpeg timeout

Note : The synchronous design means the HTTP connection stays open during inference. For v1's single-user use case, this is correct.

Request Lifecycle — Diagram



Blue = frontend, Yellow = validation, Orange = processing, Green = cleanup.

Host vs. Container Responsibilities

The boundary between host and container is drawn by hardware capability.

Host (Browser)

- Microphone access
- Audio recording
- User interface rendering
- Text editing
- Clipboard access
- Codec negotiation

Container (Docker)

- Model weight management
- Audio transcoding (ffmpeg)
- GPU inference (faster-whisper)
- MIME validation
- Temp file lifecycle
- Structured logging

Browser APIs (mic, clipboard) cannot run in a container. GPU inference cannot run efficiently in a browser. The container is disposable — only the model cache and the transcript text survive beyond a single session.

GPU Passthrough

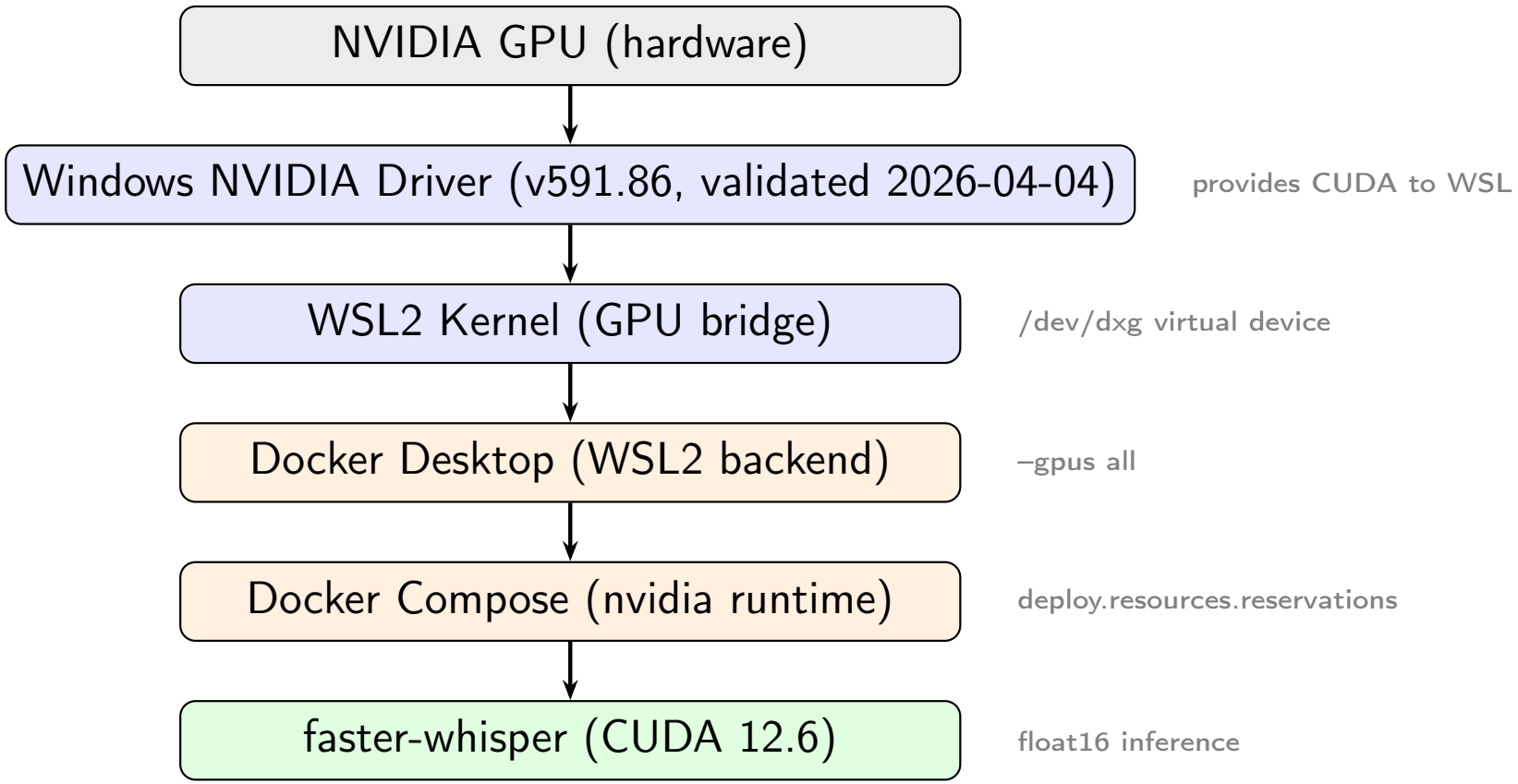
Voice Scribe uses NVIDIA GPU passthrough from Windows through four layers. Each layer must be correctly configured for the next to function.

- **Windows NVIDIA driver** provides CUDA to the WSL2 kernel
- **WSL2 kernel** bridges GPU access into Linux userspace
- **Docker Desktop** (WSL2 backend) exposes the GPU to containers
- **Docker Compose** `deploy.resources` reserves the device

This is the highest-risk setup step in the entire project. A misaligned driver version, WSL kernel, or Docker Desktop setting breaks the chain silently.

Note : The bootstrap script `ops/scripts/bootstrap_gpu_check.sh` gates all development behind GPU verification. Do not write application code until it passes.

GPU Passthrough — Diagram



Each layer depends on the one above it. The bootstrap check verifies the full chain by running `nvidia-smi` inside a GPU-enabled container.

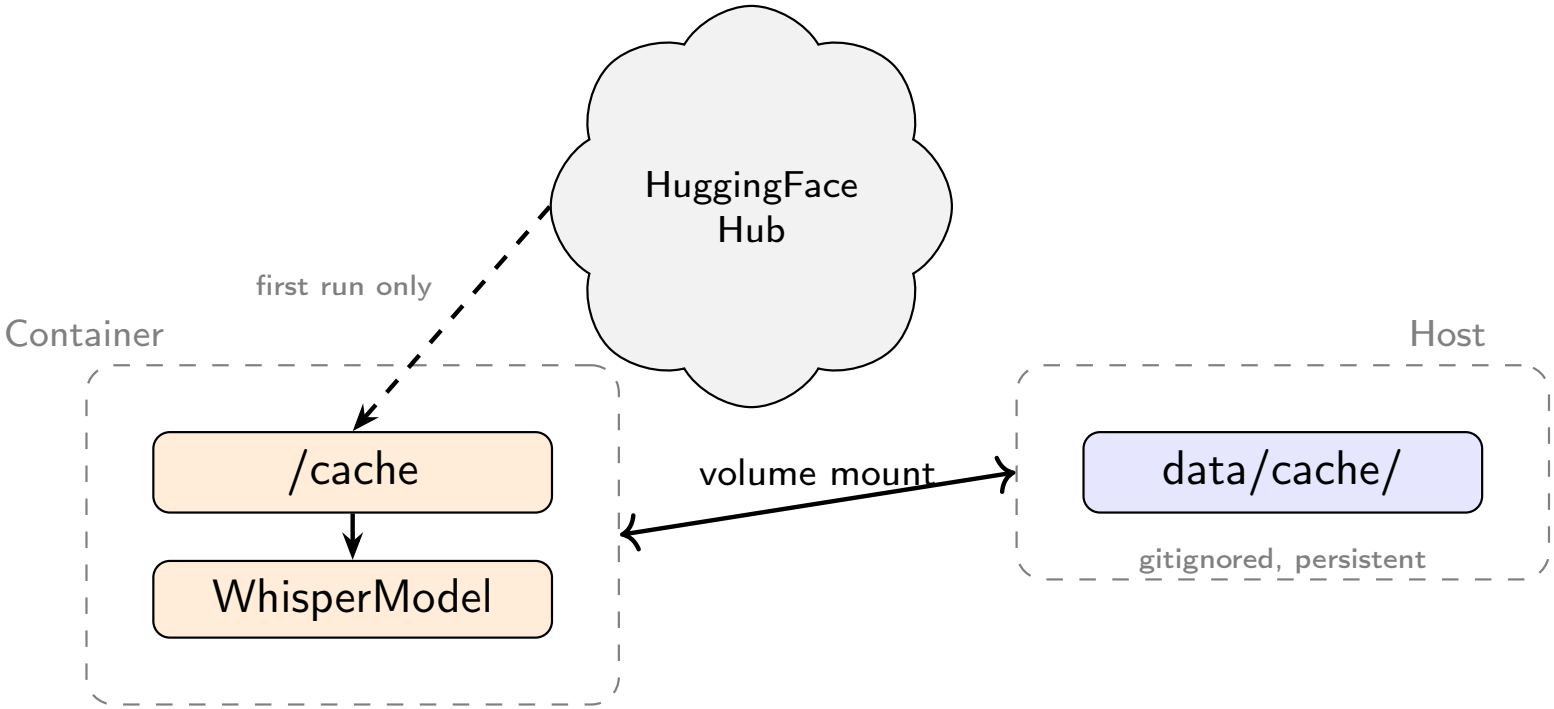
Model Caching

The `large-v3` model is ~3 GB. It downloads once from HuggingFace and must persist across container restarts.

- Model weights are **not** baked into the Docker image — this would bloat the image and prevent model swaps
- A host-mounted volume (`data/cache/`) maps to `/cache` inside the container
- First run: ~60 s download. Subsequent runs: < 2 s load
- `data/cache/` is gitignored — weights never enter the repo

Remark : The compose file sets `user: ${UID}:${GID}` so cache files are owned by the host user, avoiding permission mismatches between container and host filesystems.

Model Caching — Diagram



Dashed arrow = one-time download. Bidirectional arrow = volume mount. Model loads from host filesystem on every startup after first run.

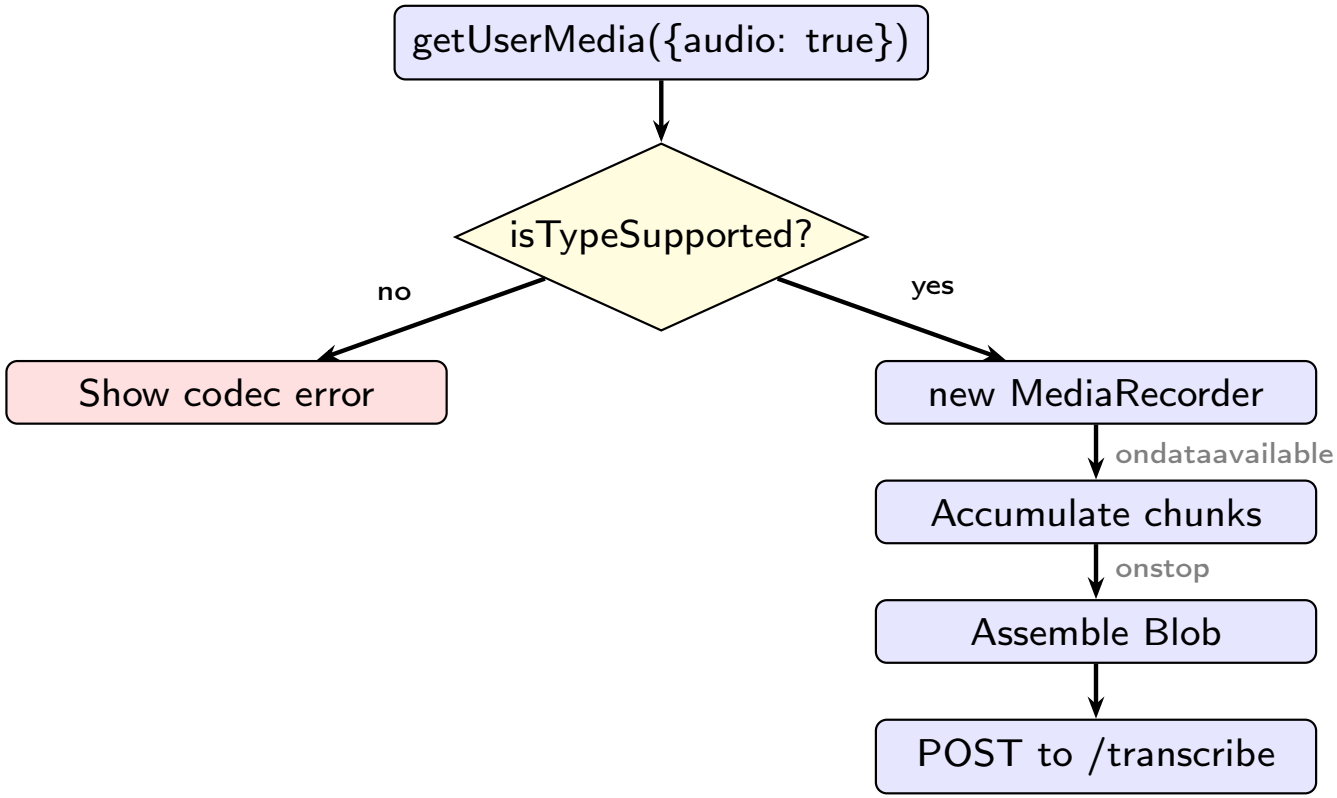
Browser Recording

Audio capture uses the Web `MediaRecorder` API, which provides raw access to the microphone through the browser.

- The recorder explicitly requests `audio/webm;codecs=opus` — never relies on browser defaults
- A pre-flight check via `isTypeSupported()` runs before any recording attempt. If the codec is unsupported, the user sees a clear error message
- Audio chunks accumulate in memory during recording and are assembled into a single Blob on stop
- Media stream tracks are cleaned up on unmount to release the microphone

Note : `MediaRecorder` does not guarantee `webm/opus` across all browsers. Chrome on Windows is the supported browser for v1. We fail explicitly on unsupported browsers rather than producing corrupt recordings.

Browser Recording — Diagram



The diamond is a hard gate — recording never starts if the codec is unsupported. This prevents silent failures where the browser records in an unexpected format that the backend cannot transcode.

Server-Side Transcoding

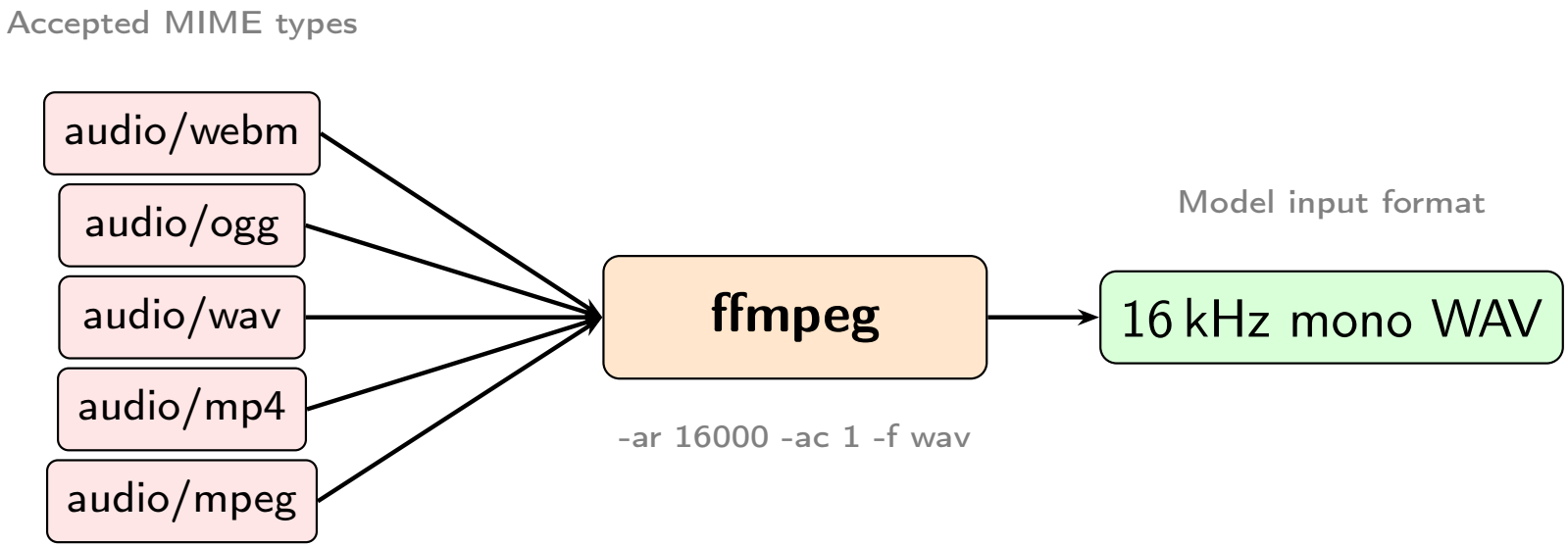
The browser produces `webm/opus`. The model expects 16 kHz mono WAV. Transcoding is performed server-side by ffmpeg inside the container.

- The backend accepts five audio MIME types — any format ffmpeg can decode is a valid input
- All inputs are normalized to the same output: 16 kHz sample rate, single channel, PCM WAV
- Adding a new input format requires only adding the MIME type to the allowlist — zero frontend changes
- ffmpeg runs with a 30-second timeout to prevent hung processes

Why Not Transcode in the Browser?

Keeping transcoding server-side means the frontend is format-agnostic. It sends whatever the `MediaRecorder` produces, and the backend normalizes. This isolates codec complexity to one place.

Server-Side Transcoding — Diagram



Five input formats funnel through a single normalization step. The model always receives identical input regardless of browser or source format. Safety bounds: 10 MB cap, 60 s duration, 30 s timeout.

Inference

`faster-whisper` provides CTranslate2-optimized Whisper inference, running locally on the GPU with no external API calls.

Model configuration

- Model: `large-v3` (default)
- Device: CUDA (GPU)
- Compute type: `float16`
- Beam size: 5
- Fallback: `small` on CPU, `int8` *(verified)*

Runtime behavior

- Loaded at startup, held in GPU memory
- Swappable at runtime (v2.0)
- Synchronous (blocks thread)
- Single-user — no concurrency
- VAD filtering enabled

Synchronous design: `faster-whisper` blocks the thread during inference. Correct for single-user. Rework path: `asyncio.to_thread()` or background task with polling (both change the API contract).

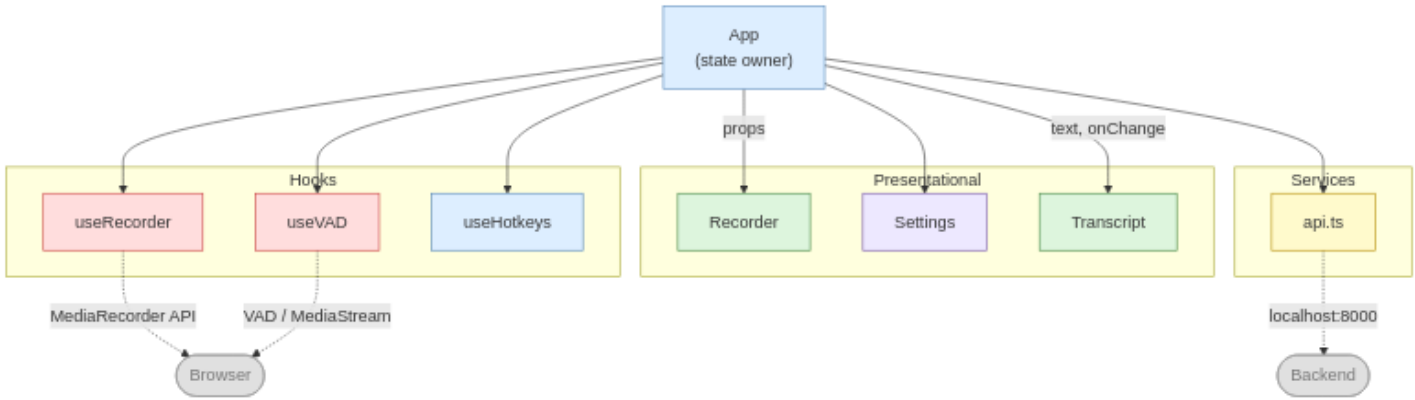
Component Architecture

The frontend is a small React application with a clear separation between container logic, presentational components, and services.

- **App** is the sole container component — it owns all state, calls `useRecorder` and `useHotkeys` directly, and orchestrates the full flow
- **Recorder**, **Transcript**, and **Settings** are presentational — they receive props and emit callbacks
- **useRecorder** and **useHotkeys** are custom hooks consumed by App, keeping browser API complexity out of components
- **api.ts** is a thin service layer — typed API calls for transcription, model list, model swap, and WebSocket streaming

Remark : No routing, no global state library, no build-time environment variables. Complexity is earned.

Component Architecture — Diagram



Blue = state owner / wiring (App, useHotkeys) Red = active-mic hooks (useRecorder, useVAD) Green = presentational components Lavender = settings overlay Yellow = service layer (api.ts) Gray = external systems. Dashed arrows = external system boundaries.

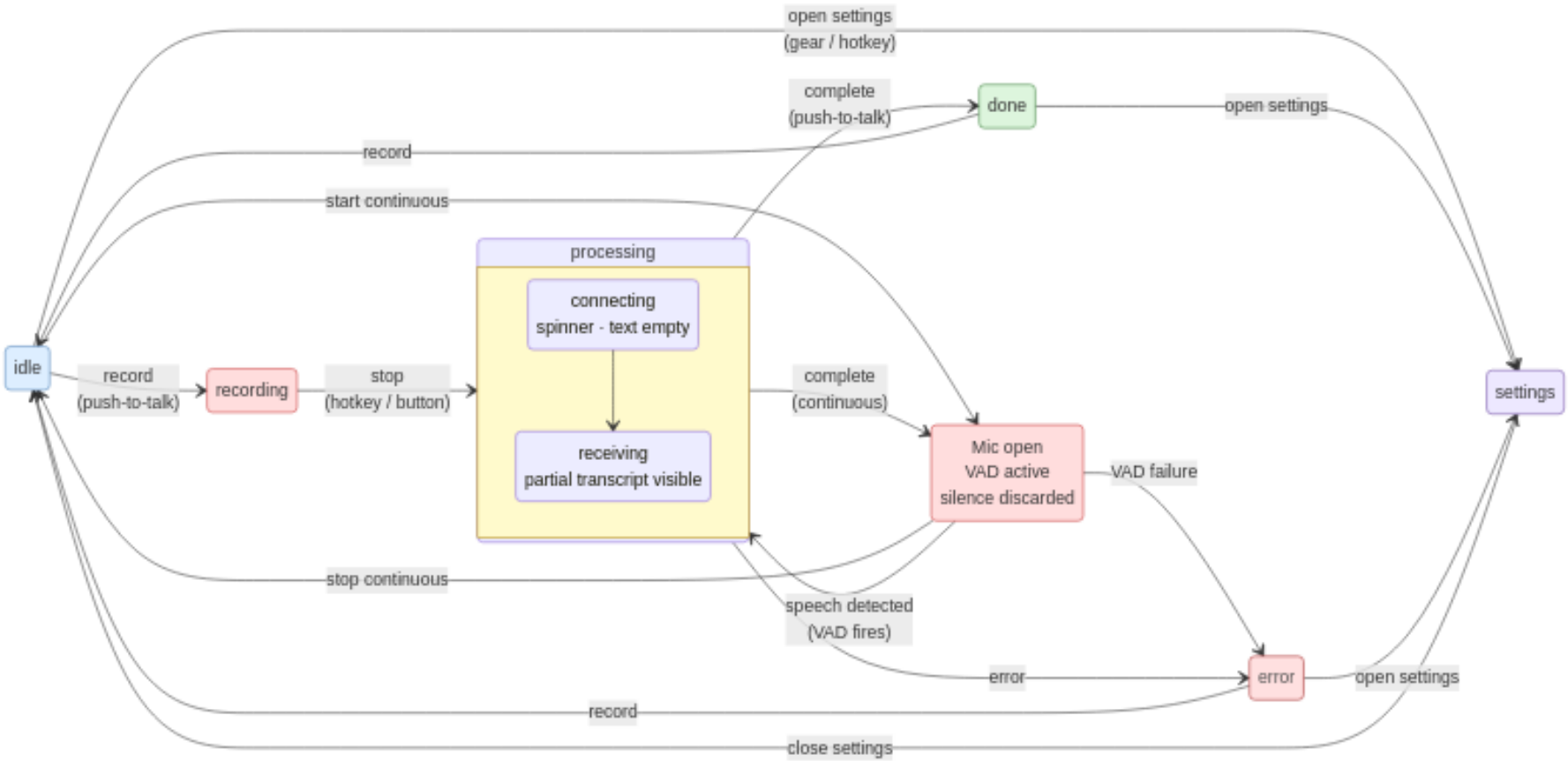
State Machine

The UI has five recording pipeline states plus a settings overlay.

- **idle** — Ready. Record button enabled.
- **recording** — Capturing audio. Button shows "Stop".
- **processing** — Transcription in progress. Button disabled. Two streaming sub-phases: *connecting* (WebSocket open, no segments yet — spinner shown) and *receiving* (segments arriving — partial transcript visible).
- **done** — Transcript in editable textarea. Copy button visible.
- **error** — Error message displayed. User can retry.

Settings overlay is orthogonal to the pipeline — a separate `settingsOpen: boolean`, not a sixth state. Accessible from idle, done, and error only. Inaccessible during recording or processing.

State Machine — Diagram



Blue = idle Red = mic active (recording / listening) Yellow = in-flight

Threat Model

Asset	Sensitivity	Location
Microphone audio	High	Browser memory, temp file (transient)
Transcript text	High	Browser memory, HTTP response
Session metadata	Low	Container logs (durations, formats)
Model weights	None	data/cache/ (public files)

Primary attack surfaces and mitigations:

- **Upload endpoint** — MIME validation, 10 MB size cap, 60 s duration cap, 30 s ffmpeg timeout
- **Temp storage** — `try/finally` cleanup; dedicated directory; crash residuals monitored
- **CORS / origin** — Restricted to localhost origins only; no wildcard
- **Dependencies** — Pinned versions, weekly Dependabot scans

Note : Most likely failure: accidental LAN exposure via bind address change.

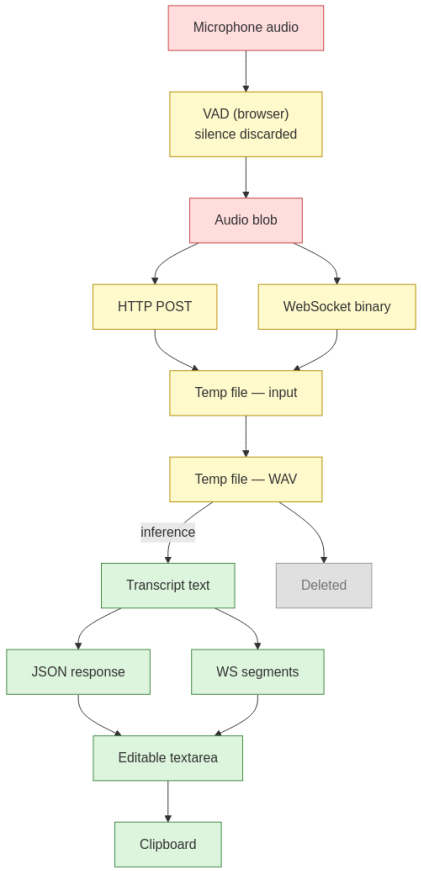
Privacy — Data Flow

- Audio exists in three places: browser memory, localhost transit (HTTP POST or WebSocket binary), and a temp file (seconds). Both transports are localhost only.
- Transcript exists in two places: localhost transit (JSON response or WebSocket segment stream) and the browser textarea. Retention is identical for both transport paths.
- **Nothing is persisted server-side** — API is stateless between requests.
- Logs contain only durations, formats, and detected language — never content.

Network Boundary

API binds to `localhost:8000`. CORS restricted to localhost origins. WebSocket origin enforced in handler (CORS middleware does not cover WS). Zero external network traffic.

Privacy — Data Flow Diagram



Red = sensitive audio (transient). Yellow = ephemeral intermediate. Green = transcript output (user-controlled). Gray = deleted. Both transports are localhost only.

Operational Telemetry

The system logs *operations*, not *content*. Every event includes an ISO 8601 timestamp via `structlog`.

Event	Fields
`model download started`	model name, device
`model download complete`	duration
`transcode started`	input MIME type
`transcode complete`	output format, duration
`inference started`	audio file path
`inference complete`	duration, detected language
`model swap started`	previous model, new model
`model swap complete`	duration

- All output is structured and greppable from container logs
- Log level defaults to `info`; adjustable via `LOG_LEVEL`
- Transcript content is **never** logged at any level — this is a hard requirement, not a convention

Workflow & Tooling

Development environment: WSL2 Linux filesystem, VS Code with WSL extension, Docker Desktop with WSL2 backend, Chrome for testing.

Makefile targets:

Command	Purpose
`make dev`	Start API container + frontend dev server
`make build`	Build the API Docker image
`make test`	Run all tests (pytest + vitest)
`make gpu-check`	Verify GPU/Docker readiness

Smoke test (fastest full-stack feedback loop):

```
bash ops/scripts/smoke_test.sh ops/scripts/test.mp3
```

Tests `GET /health`, `POST /transcribe`, and `GET /models` against the running API.

Testing Strategy

Backend (pytest, 41 tests)

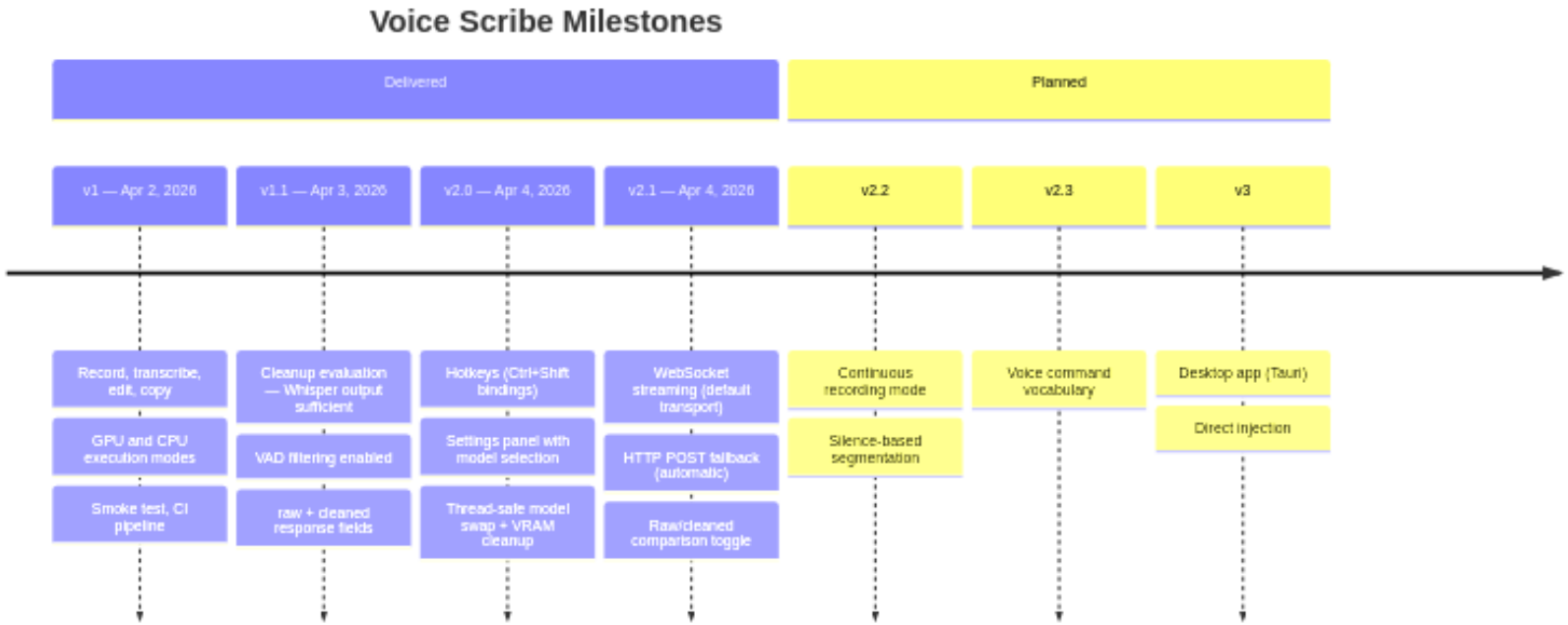
- Health endpoint responses
- MIME validation + codec stripping
- File size rejection
- Happy-path transcription
- Model list and swap endpoints
- WebSocket endpoint: origin enforcement, size validation, happy path, inference failure, swap concurrency contract

Frontend (vitest, 22 tests)

- Recorder presentational props
- Transcript rendering and copy
- Settings panel: model list, swap, loading, error, retry
- Settings display: comparison toggle renders, fires correct callback

Philosophy: Happy path plus obvious failure boundaries. Smoke test exercises the running system end-to-end (`/health`, `/transcribe`, `/models`).

Milestone Timeline



Delivered milestones show actual dates and duration. Planned milestones show sequence only — no hard deadlines. The project iterates as it continues; scope is earned, not scheduled.

Roadmap Detail

v2.0 (complete)

- Hotkey system
- Settings panel (model selection)
- State machine diagram update

v2.1 (complete)

- WebSocket streaming (default transport)
- HTTP POST fallback (automatic)
- Raw/cleaned comparison toggle

v2.2

- Continuous recording mode
- Client-side VAD (`@ricky0123/vad-web`)
- Silence-based segmentation

v2.3

- Voice command vocabulary
- Stop, submit, redo, clear phrases

v3 — Desktop packaging (Tauri). Global hotkeys. Direct injection into focused application. No Docker dependency.

v2.0 — Hotkeys + Settings

Hotkey system — keyboard shortcuts for recording control, copy, clear, and settings toggle. All use `Ctrl+Shift+<key>` modifier to avoid conflicts with the editable textarea and Chrome built-in bindings.

Hotkey	Action
`Ctrl+Shift+Space`	Toggle recording
`Ctrl+Shift+Y`	Copy transcript
`Ctrl+Shift+X`	Clear transcript
`Ctrl+Shift+S`	Toggle settings panel

Settings panel — model selection (swap between `large-v3` and `small` at runtime without editing `.env`). Model swap is full-stack: backend endpoint with threading lock, VRAM cleanup, and graceful fallback.

v2.0 — Model Swap Architecture

Runtime model selection requires a backend endpoint and safe reload logic.

- `GET /models` — returns available models (configured via `WHISPER_AVAILABLE_MODELS` env var)
- `POST /models/swap` — accepts model name, acquires threading lock, loads new model, frees old VRAM
- Swap blocks inference during reload (seconds). Frontend shows loading state; record button disabled
- If swap fails, old model remains loaded — no degradation
- Regular `def` endpoint (not `async def`) — runs in thread pool, keeps event loop alive for health checks

Available Models

Intentionally limited to models declared in `WHISPER_AVAILABLE_MODELS`. No auto-discovery, no surprise downloads.

v2.1 — WebSocket Streaming

Recording flow unchanged — start, speak, stop. What changes is delivery.

- Client connects to `ws://localhost:8000/ws/transcribe`
- Full audio blob sent as binary after recording stops
- Server streams `{"type":"segment"}` messages as faster-whisper yields each segment
- Final `{"type":"complete"}` mirrors `TranscribeResponse` — raw, cleaned, language, duration, model, device
- Partial text accumulates in the textarea during inference

Note on GPU hardware

An RTX 3090 running `large-v3` finishes 30s of audio in ≈2.5s — segments arrive in a burst, indistinguishable from batch. Streaming is perceptible on CPU or in v2.2 continuous mode.

v2.1 — Transport Architecture

Event	Cause	Response
`ws.onerror`	Server unreachable, network dropped	Fall back to `POST /transcribe`
`{type: "error"}`	Inference or transcode failed	Surface immediately — no fallback

Enforced constraints

- Origin checked against `CORS_ORIGINS` before `accept()` — CORS middleware does not cover WebSocket
- Size validated after `receive_bytes()` — same `MAX_UPLOAD_MB` limit as HTTP route
- ffmpeg runs via `asyncio.to_thread()` — non-blocking, 30s timeout
- Temp files cleaned in `try/finally` on all exit paths

v2.1 — Concurrency Contract

`swap_model()` can fire mid-segment-iteration. No new locks needed.

Why it is safe

- `run_inference()` captures `model = get_model()` at thread start — a strong Python reference
- `swap_model()` replaces `_model` and calls `del old_model` — removes swap's local reference only
- Python's refcount cannot reach zero while the inference thread's `model` variable is alive
- `gc.collect()` and `torch.cuda.empty_cache()` are benign against a live reference — no in-use tensors freed
- VRAM cleanup deferred until inference thread completes naturally

Model name

Captured alongside `get_model()` at thread start. Reflects the model that ran inference, not the model loaded at message-send time.